

Министерство образования Республики Беларусь  
Учреждение образования «Белорусский государственный  
университет информатики и радиоэлектроники»  
Факультет «Информационных технологий и управления»

## ОТЧЕТ

По лабораторной работе №6  
По дисциплине «Аппаратное обеспечение интеллектуальных систем»  
**Моделирование таблиц хеширования**

Выполнил: студент гр. 421702  
Дождиков А. И.

Проверил: доцент, Каф. ИИТ  
Захаров В. В.

# 1 Теоретические сведения

Таблицы хеширования применяются для обеспечения программного поиска информации по содержанию специального ключевого слова или фрагмента самих данных.

Основные компоненты:

- **ID** – ключевое слово или его идентификатор;
- **V** – числовое значение ключевого слова;
- **h(V)** – хеш-адрес, вычисленный по V;
- **H** – общее количество строк в таблице; **B** – начальный адрес.

Числовое значение V вычисляется по первым двум символам ключевого слова в позиционной системе счисления с основанием 33 (русский алфавит):

$$V = N_1 \cdot 33^1 + N_2 \cdot 33^0$$

Хеш-адрес определяется формулой:  $h(V) = (V \bmod H + B) \bmod H$ .

При совпадении адресов возникает **коллизия**. Для её разрешения используется линейный пробинг с внутренней адресацией. Структура ячейки содержит флаги: **C** (коллизия), **U** (занято), **T** (терминальный), **L** (связь), **D** (удалено), а также указатель **P0** на следующий элемент цепочки и поле данных **Pi**.

Удаление выполняется путём анализа флажков **T** и **C** с установкой **D=1** и перестройкой цепочки в соответствии с правилами, описанными в методических указаниях.

## 2 Исходные данные

### Вариант 9: Тематическая направленность "Спорт"

Размер таблицы:  $H = 20$ . Начальный адрес:  $B = 0$ .

Ключевые слова (ID) и данные (Pi) приведены в таблице 1.

Таблица 1: Исходные данные для формирования хеш-таблицы

| №  | ID (Спорт) | Pi (Определение)              | V   | h(V) |
|----|------------|-------------------------------|-----|------|
| 1  | Футбол     | Командная игра с мячом        | 713 | 13   |
| 2  | Баскетбол  | Игра с мячом и корзиной       | 33  | 13   |
| 3  | Теннис     | Индивидуальный или парный вид | 632 | 12   |
| 4  | Волейбол   | Игра через сетку              | 81  | 1    |
| 5  | Хоккей     | Зимний командный вид спорта   | 741 | 1    |
| 6  | Плавание   | Преодоление дистанции в воде  | 540 | 0    |
| 7  | Бокс       | Единоборство в перчатках      | 48  | 8    |
| 8  | Дзюдо      | Японское боевое искусство     | 140 | 0    |
| 9  | Самбо      | Советское боевое искусство    | 594 | 14   |
| 10 | Гимнастика | Упражнения на снарядах        | 108 | 8    |
| 11 | Гребля     | Спорт на водной глади         | 116 | 16   |
| 12 | Парусный   | Управление судном парусами    | 528 | 8    |

Количество коллизий: 4. Количество цепочек резервных ячеек: 4. Требования методики соблюдены.

### 3 Формирование хеш-таблицы

Формирование осуществлялось методом линейного пробинга. При занятости ячейки  $h(V)$  производился последовательный поиск свободного места по адресам  $(h+1)\%H, (h+2)\%H \dots$ . Флаги **C**, **T**, **P0** обновлялись для поддержания структуры цепочек.

Таблица 2: Состояние хеш-таблицы после заполнения

| №  | ID         | C | U | T | L | D | P0 | V   | h  |
|----|------------|---|---|---|---|---|----|-----|----|
| 0  | Плавание   | 0 | 1 | 0 | 0 | 0 | 3  | 540 | 0  |
| 1  | Волейбол   | 0 | 1 | 0 | 0 | 0 | 2  | 81  | 1  |
| 2  | Хоккей     | 1 | 1 | 1 | 0 | 0 | 2  | 741 | 1  |
| 3  | Дзюдо      | 1 | 1 | 1 | 0 | 0 | 3  | 140 | 0  |
| ⋮  | ⋮          | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮  | ⋮   | ⋮  |
| 8  | Бокс       | 0 | 1 | 0 | 0 | 0 | 9  | 48  | 8  |
| 9  | Гимнастика | 1 | 1 | 0 | 0 | 0 | 10 | 108 | 8  |
| 10 | Парусный   | 1 | 1 | 1 | 0 | 0 | 10 | 528 | 8  |
| ⋮  | ⋮          | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮  | ⋮   | ⋮  |
| 12 | Теннис     | 0 | 1 | 1 | 0 | 0 | 12 | 632 | 12 |
| 13 | Футбол     | 0 | 1 | 0 | 0 | 0 | 14 | 713 | 13 |
| 14 | Баскетбол  | 1 | 1 | 0 | 0 | 0 | 15 | 33  | 13 |
| 15 | Самбо      | 1 | 1 | 1 | 0 | 0 | 15 | 594 | 14 |
| 16 | Гребля     | 0 | 1 | 1 | 0 | 0 | 16 | 116 | 16 |

### 4 Реализация операций

Программа реализует полный цикл работы с таблицей:

1. **Поиск:** вычисляется  $h(V)$ , осуществляется последовательный проход по цепочке. Найденный элемент возвращает данные **Pi**.
2. **Добавление:** проверяется уникальность ключа. При коллизии используется линейный пробинг, обновляются флаги **C=1**, **T=0** для предыдущего узла и **P0**.
3. **Удаление:** устанавливается **D=1**. В зависимости от комбинации **T** и **C** выполняется перестройка цепочки (сдвиг элементов или сброс **U=0**) согласно алгоритму из методички.
4. **Коэффициент заполнения:**  $\alpha = N_{occupied}/H$ . Для исходных данных  $\alpha = 12/20 = 0.60$ .

### 5 Выводы

В ходе выполнения лабораторной работы №6 были освоены навыки построения и проверки таблиц хеширования. Для варианта 9 (Спорт) выполнено следующее:

- Спроектирована структура ячейки с флагами **C**, **U**, **T**, **L**, **D**, **P0**, **Pi**;
- Реализовано вычисление хеш-адресов по первым двум буквам ключа в системе счисления с основанием 33;

- Настроен механизм обработки коллизий методом линейного пробинга с формированием цепочек резервных ячеек;
- Разработан алгоритм удаления с корректным обновлением указателей и флагов;
- Коэффициент заполнения таблицы составил 0.60, что обеспечивает эффективный поиск.

Все требования методических указаний соблюдены. Программа успешно протестирована на наборе из 12 записей.

## 6 Листинг кода

Листинг 1: Программа моделирования хеш-таблицы (Вариант 9)

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  import sys
4
5  ALPHABET = "....."
6  BASE = 33
7
8  class HashCell:
9      def __init__(self):
10         self.ID = ""
11         self.C = 0
12         self.U = 0
13         self.T = 1
14         self.L = 0
15         self.D = 0
16         self.P0 = 0
17         self.Pi = ""
18
19  class HashTable:
20      def __init__(self, H=20, B=0):
21         self.H = H
22         self.B = B
23         self.table = [HashCell() for _ in range(H)]
24
25      def calc_v(self, keyword: str) -> int:
26         if len(keyword) < 2:
27             return 0
28         i1 = ALPHABET.find(keyword[0].upper())
29         i2 = ALPHABET.find(keyword[1].upper())
30         if i1 == -1 or i2 == -1:
31             return 0
32         return i1 * BASE + i2
33
34      def calc_h(self, V: int) -> int:
35         return (V % self.H + self.B) % self.H
36
37      def insert(self, keyword: str, data: str) -> bool:
38         idx, _ = self.search(keyword)

```

```

39     if idx != -1:
40         print(f"[WARN] ~~~~~~ '{keyword}' ~~~~~~.")
41         return False
42
43     V = self.calc_v(keyword)
44     h = self.calc_h(V)
45     idx = h
46     steps = 0
47
48     while self.table[idx].U == 1 and steps < self.H:
49         idx = (idx + 1) % self.H
50         steps += 1
51
52     if steps == self.H:
53         print("[ERROR] ~~~~~~.")
54         return False
55
56     cell = self.table[idx]
57     cell.ID = keyword
58     cell.Pi = data
59     cell.U = 1
60     cell.D = 0
61     cell.P0 = idx
62
63     if idx != h:
64         cell.C = 0
65         curr = h
66         prev_idx = -1
67         while curr != idx:
68             if self.table[curr].U == 1 and self.table[curr].D == 0:
69                 prev_idx = curr
70                 curr = (curr + 1) % self.H
71
72             if prev_idx != -1:
73                 self.table[prev_idx].P0 = idx
74                 self.table[prev_idx].T = 0
75             self.table[h].C = 1
76     else:
77         cell.C = 0
78         cell.T = 1
79
80     print(f"[OK] ~~~~~~: '{keyword}' ~~~~~~ {idx} (h={h})")
81     return True
82
83 def search(self, keyword: str) -> tuple:
84     V = self.calc_v(keyword)
85     h = self.calc_h(V)
86     idx = h
87     steps = 0
88     while steps < self.H:
89         cell = self.table[idx]
90         if cell.U == 1 and cell.D == 0 and cell.ID == keyword:

```

```

91         return idx, cell.Pi
92     if cell.U == 0 or (cell.T == 1 and cell.P0 == idx):
93         break
94     idx = (idx + 1) % self.H
95     steps += 1
96     return -1, None
97
98 def delete(self, keyword: str) -> bool:
99     idx, _ = self.search(keyword)
100     if idx == -1:
101         print("[WARN] .....")
102         return False
103
104     cell = self.table[idx]
105     h_del = self.calc_h(self.calc_v(keyword))
106     cell.D = 1
107
108     # `) .....
109     if cell.T == 1 and cell.P0 == idx:
110         cell.U = 0
111     # `) .....
112     elif cell.T == 1 and cell.P0 != idx:
113         cell.U = 0
114         curr = h_del
115         while curr != idx:
116             if self.table[curr].U == 1 and self.table[curr].D == 0
117                 and self.table[curr].P0 == idx:
118                 self.table[curr].T = 1
119                 break
120             curr = (curr + 1) % self.H
121     # `) ..... (T=0, P0 != h_del)
122     elif cell.T == 0 and cell.P0 != h_del:
123         next_idx = cell.P0
124         nxt = self.table[next_idx]
125         cell.ID, cell.Pi, cell.C, cell.T, cell.L, cell.P0 = \
126             nxt.ID, nxt.Pi, nxt.C, nxt.T, nxt.L, nxt.P0
127         nxt.U = 0
128         nxt.D = 1
129     # `) ..... (T=0, C=1)
130     elif cell.T == 0 and cell.C == 1:
131         next_idx = cell.P0
132         nxt = self.table[next_idx]
133         cell.ID, cell.Pi, cell.C, cell.T, cell.L, cell.P0 = \
134             nxt.ID, nxt.Pi, nxt.C, nxt.T, nxt.L, nxt.P0
135         nxt.U = 0
136         nxt.D = 1
137
138     print(f"[OK] .....: '{keyword}' (..... {idx})")
139     return True
140
141 def display(self):
142     # .....

```

```

142     header = f"{'':<5} | {'ID':<16} | {'C':<3} | {'U':<3} | {'T'
        ':<3} | {'L':<3} | {'D':<3} | {'P0':<4} | {'Pi':<35} | {'V'
        ':<6} | {'h':<4}"
143     print(header)
144     print("-" * len(header))
145
146     for i, c in enumerate(self.table):
147         if c.ID:
148             V = self.calc_v(c.ID)
149             h = self.calc_h(V)
150         else:
151             V = "-"
152             h = "-"
153
154         # ..... , .....
155
156         pi_str = str(c.Pi)[:35].ljust(35)
157         print(f"{'i':<5} | {'c.ID':<16} | {'c.C':<3} | {'c.U':<3} | {'c.T'
            ':<3} | {'c.L':<3} | {'c.D':<3} | {'c.P0':<4} | {pi_str} | {str
            (V):<6} | {str(h):<4}")
158
159         occupied = sum(1 for c in self.table if c.U == 1 and c.D == 0)
160         print(f"[STAT] .....: {occupied / self.H:.2%}"
            )
161
162 def main():
163     ht = HashTable(H=20, B=0)
164
165     initial_data = [
166         (".....", "....."),
167         (".....", "....."),
168         (".....", "....."),
169         (".....", "....."),
170         (".....", "....."),
171         (".....", "....."),
172         (".....", "....."),
173         (".....", "....."),
174         (".....", "....."),
175         (".....", "....."),
176         (".....", ".....")
177     ]
178
179     print("[INIT] ..... (..... 9: .....)...")
180     for kw, data in initial_data:
181         ht.insert(kw, data)
182     ht.display()
183
184     print("\n[SEARCH] .....: '.....'")
185     idx, val = ht.search(".....")
186     print(f".....: ..... {idx}, .....: {val}" if idx != -1 else "
        .....")
187

```

